Módulos integrados

Mario Hernando Gallo Gallego

GAES # 6

Servicio Nacional de Aprendizaje – (SENA)

Análisis y Desarrollo de Software – (3118493)

Andrés Rubiano Cucarían

Cali, Colombia

22 de marzo de 2026

1. INTRODUCCIÓN

El presente documento describe el proceso de desarrollo, integración y validación de los módulos que componen el sistema de gestión de la empresa Proterquim. Este sistema fue diseñado bajo un enfoque modular y una arquitectura cliente-servidor, permitiendo la separación de responsabilidades entre la capa de presentación (frontend), la lógica del negocio (backend) y la capa de datos (base de datos).

Se implementó el módulo de gestión de empleados como componente funcional principal, el cual permite realizar operaciones CRUD (Crear, Listar, Actualizar y Eliminar) sobre los registros almacenados. Adicionalmente, se definieron otros módulos como parte de la arquitectura general del sistema, los cuales podrán ser implementados en futuras iteraciones.

Este desarrollo se realizó siguiendo los lineamientos del componente formativo, aplicando buenas prácticas de programación, control de versiones y documentación técnica del software.

2. OBJETIVO

Objetivo general

Desarrollar e integrar los módulos del sistema de información de Proterquim, garantizando su correcto funcionamiento mediante la implementación de una arquitectura modular.

Objetivos específicos

- ✓ Implementar el módulo de gestión de empleados.
- ✓ Integrar frontend, backend y base de datos.
- ✓ Documentar los módulos del sistema y su interacción.
- ✓ Validar el funcionamiento mediante pruebas CRUD.
- ✓ Gestionar el código fuente mediante repositorio Git.

3. ALCANCE DEL PROYECTO

El presente proyecto contempla el desarrollo e integración del módulo de gestión de empleados dentro del sistema de información de la empresa Proterquim, permitiendo realizar operaciones CRUD (Crear, Listar, Actualizar y Eliminar) mediante una aplicación web.

El sistema integra un frontend desarrollado en Angular, un backend en Node.js con Express y una base de datos MongoDB, garantizando la comunicación entre sus componentes a través de servicios REST.

Asimismo, se define la arquitectura general del sistema bajo un enfoque modular, donde se identifican otros módulos como gestión de usuarios, productos e inventarios, los cuales son considerados como parte del diseño del sistema para futuras implementaciones.

Incluye:

- ✓ Desarrollo del módulo de gestión de empleados
- ✓ Integración entre frontend, backend y base de datos
- ✓ Implementación de operaciones CRUD
- ✓ Consumo de API REST
- ✓ Validaciones básicas en el sistema
- ✓ Pruebas funcionales del módulo
- ✓ Documentación técnica del sistema
- ✓ Uso de repositorio Git para control de versiones

No incluye:

- ✓ Implementación del módulo de autenticación (login y registro)
- ✓ Desarrollo completo de módulos de productos e inventarios
- ✓ Despliegue en servidores en la nube
- ✓ Implementación de roles y permisos de usuario
- ✓ Pruebas automatizadas unitarias

4. ACTAS DE APROBACIÓN DE REQUERIMIENTOS

Durante el desarrollo del sistema, los requerimientos funcionales y no funcionales fueron definidos y validados previamente, asegurando que el sistema cumpla con las necesidades planteadas.

Para esta evidencia, se consideran como soporte las actas de aprobación de requerimientos elaboradas en fases anteriores del proyecto, las cuales incluyen la definición del sistema de inventarios y sus módulos asociados.

Estas actas permiten garantizar la trazabilidad del proyecto y la correcta alineación entre los requerimientos y el desarrollo del sistema.

ACTA DE APROBACIÓN DE REQUERIMIENTOS

Proyecto: Sistema de Gestión Proterquim

Módulo: Gestión de Empleados

Fecha: 22 de marzo de 2026

Responsable: Mario Hernando Gallo Gallego

1. Objetivo

Validar y aprobar los requerimientos funcionales y no funcionales del sistema, asegurando que cumplen con las necesidades del usuario y los objetivos del proyecto.

2. Requerimientos Funcionales

- ✓ RF01: Registrar empleados
- ✓ RF02: Consultar empleados
- ✓ RF03: Actualizar empleados
- ✓ RF04: Eliminar empleados

3. Requerimientos No Funcionales

- ✓ RNF01: El sistema debe ser accesible desde navegador web
- ✓ RNF02: El sistema debe responder en menos de 2 segundos
- ✓ RNF03: Interfaz amigable e intuitiva
- ✓ RNF04: Disponibilidad del sistema en entorno local

4. Observaciones

Los requerimientos definidos cumplen con las necesidades básicas del sistema y permiten el desarrollo del módulo de gestión de empleados.

5. Aprobación

Se aprueban los requerimientos definidos para el desarrollo del sistema.

Firma Responsable: Mario Gallo

5. DESCRIPCIÓN DEL SISTEMA

El sistema desarrollado corresponde a una aplicación web orientada a la gestión de empleados de la empresa Proterquim. Su objetivo principal es facilitar el registro, consulta, actualización y eliminación de información mediante una interfaz amigable e intuitiva.

El sistema permite:

- ✓ Registrar nuevos empleados con datos como nombre, cargo y salario
- ✓ Consultar el listado de empleados almacenados
- ✓ Actualizar la información de empleados existentes
- ✓ Eliminar registros de manera segura

Este sistema fue implementado bajo una arquitectura cliente-servidor, donde el frontend se encarga de la interacción con el usuario, el backend de la lógica del negocio y la base de datos del almacenamiento de la información.

6. ARQUITECTURA DEL SISTEMA

El sistema fue desarrollado bajo una arquitectura de tres capas, la cual permite una adecuada separación de responsabilidades y facilita la escalabilidad y mantenibilidad del software.

Capa de Presentación (Frontend)

Implementada en Angular, se encarga de la interacción con el usuario, captura de datos y visualización de la información mediante componentes reutilizables.

Capa de Lógica de Negocio (Backend)

Desarrollada con Node.js y Express, se encarga de procesar las solicitudes del usuario, validar la información y exponer los servicios REST que son consumidos por el frontend.

Capa de Datos (Base de Datos)

Implementada con MongoDB, permite el almacenamiento persistente de la información de los empleados, garantizando integridad y disponibilidad de los datos.

7. MÓDULOS DEL SISTEMA

Siguiendo el enfoque modular del material de estudio, el sistema se compone de:

7.1 Módulo de Gestión de Empleados (IMPLEMENTADO)

Permite realizar operaciones CRUD:

- ✓ Crear empleado
- ✓ Listar empleados
- ✓ Actualizar empleado

- ✓ Eliminar empleado

7.2 Módulo de Gestión de Usuarios (PROPUESTO)

Encargado de:

- ✓ Registro de usuarios
- ✓ Inicio de sesión
- ✓ Control de acceso

7.3 Módulo de Productos (PROPUESTO)

Permite:

- ✓ Registro de productos
- ✓ Consulta
- ✓ Actualización

7.4 Módulo de Inventarios (PROPUESTO)

Permite:

- ✓ Control de stock
- ✓ Movimientos de inventario

- ✓ Alertas de inventario

8. INTEGRACIÓN DE MÓDULOS

La integración de módulos en el sistema se realiza mediante la comunicación entre el frontend y el backend a través de servicios REST. El frontend consume los endpoints definidos en el backend utilizando peticiones HTTP (GET, POST, PUT y DELETE), lo que permite la interacción con la base de datos.

Este modelo de integración garantiza:

- ✓ Separación de responsabilidades
- ✓ Reutilización de servicios
- ✓ Escalabilidad del sistema
- ✓ Facilidad de mantenimiento

La base de datos MongoDB actúa como repositorio central de información, permitiendo que los diferentes módulos del sistema puedan interactuar de manera eficiente.

9. REPOSITORIO DE CÓDIGO FUENTE

El proyecto se encuentra alojado en el siguiente repositorio:

- ✓ https://github.com/MG2079/proterquim-empleados
- ✓ Se utilizó Git como sistema de control de versiones.

10. EJECUCIÓN DEL SISTEMA

Backend:

- ✓ npm start

Frontend:

- ✓ ng serve

11. URL DEL SISTEMA

- ✓ Frontend: http://localhost:4200
- ✓ Backend: http://localhost:3000/api/empleados

12. PRUEBAS REALIZADAS

Se realizaron pruebas funcionales con el objetivo de validar el correcto funcionamiento del módulo de gestión de empleados. Estas pruebas se llevaron a cabo mediante el consumo de los servicios REST y la verificación de los resultados en la interfaz del sistema.

Se evaluaron las siguientes operaciones:

- ✓ Creación de empleados mediante peticiones POST
- ✓ Consulta de empleados mediante peticiones GET
- ✓ Actualización de registros mediante peticiones PUT

- ✓ Eliminación de empleados mediante peticiones DELETE

Los resultados obtenidos fueron satisfactorios, evidenciando la correcta integración entre frontend, backend y base de datos.

13. CONFIGURACIÓN DEL ENTORNO

Herramientas utilizadas:

- ✓ Visual Studio Code
- ✓ Node.js
- ✓ Angular CLI
- ✓ MongoDB

Configuración de servidores:

- ✓ Backend: puerto 3000
- ✓ Frontend: puerto 4200

14. MANUAL TÉCNICO

1. Descripción General

El sistema corresponde a una aplicación web desarrollada para la gestión de empleados, implementada bajo una arquitectura cliente-servidor utilizando Angular, Node.js y MongoDB.

2. Requisitos del Sistema

Software requerido:

- ✓ Node.js
- ✓ Angular CLI
- ✓ MongoDB
- ✓ Visual Studio Code
- ✓ Navegador web (Google Chrome recomendado)

3. Estructura del Proyecto

El sistema se organiza en:

- ✓ frontend-angular: interfaz de usuario
- ✓ backend-node: lógica del negocio y API REST
- ✓ base de datos: MongoDB

4. Instalación

Clonar el repositorio:

git clone https://github.com/MG2079/proterquim-empleados

Instalar dependencias backend:

- ✓ cd backend-node
- ✓ npm install

Instalar dependencias frontend:

- ✓ cd frontend-angular
- ✓ npm install

5. Ejecución del Sistema

Backend:

- ✓ npm start

Frontend:

- ✓ ng serve

6. Configuración

- ✓ Backend: puerto 3000
- ✓ Frontend: puerto 4200
- ✓ Base de datos: MongoDB local

7. Endpoints del Sistema

- ✓ GET /api/empleados
- ✓ POST /api/empleados
- ✓ PUT /api/empleados/:id
- ✓ DELETE /api/empleados/:id

8. Consideraciones Técnicas

- ✓ El sistema utiliza formato JSON para intercambio de datos
- ✓ Se implementa arquitectura modular
- ✓ Comunicación mediante HTTP

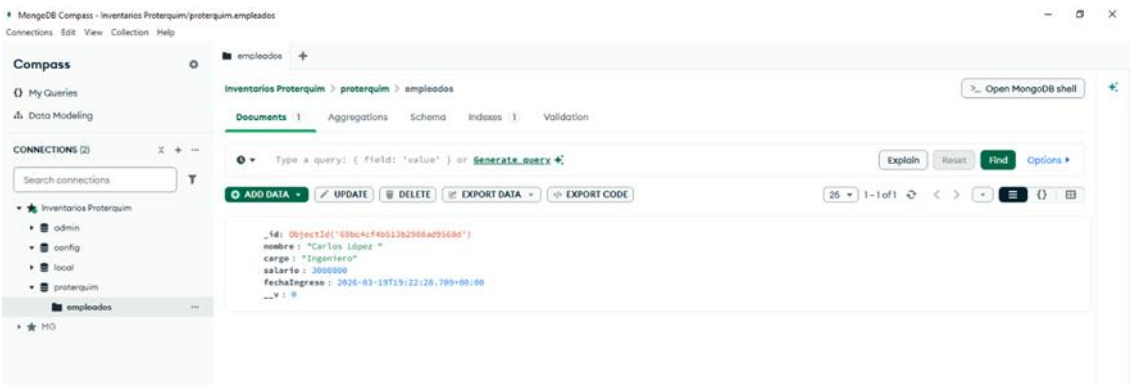
15. ANEXOS

1. EJECUCIÓN DEL BACKEND (SERVIDOR)

Descripción:

Se evidencia la ejecución del servidor backend desarrollado en Node.js, el cual se encuentra activo en el puerto 3000 y conectado a la base de datos MongoDB.

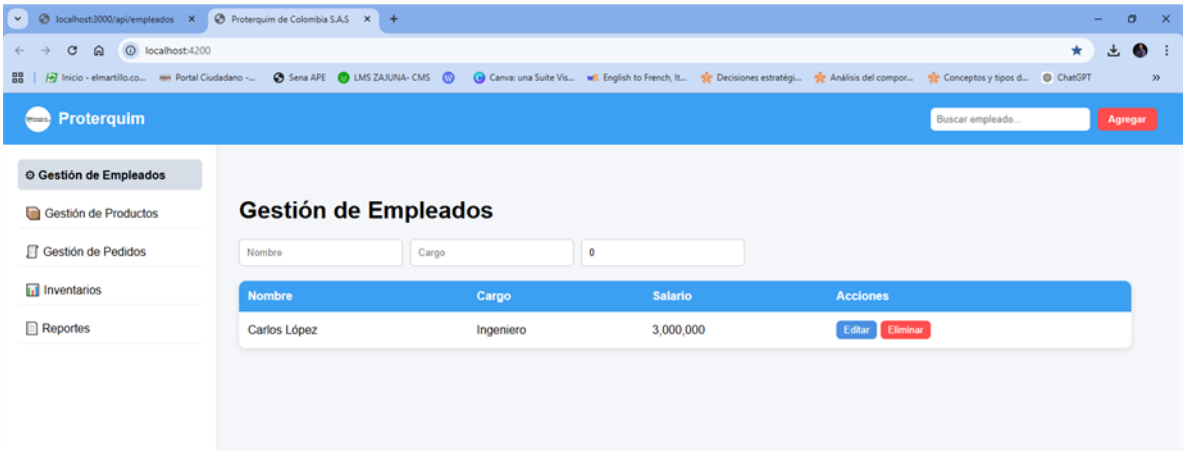
2. BASE DE DATOS (MongoDB)



Descripción:

Se muestra la base de datos MongoDB donde se almacenan los registros de empleados, evidenciando la persistencia de la información.

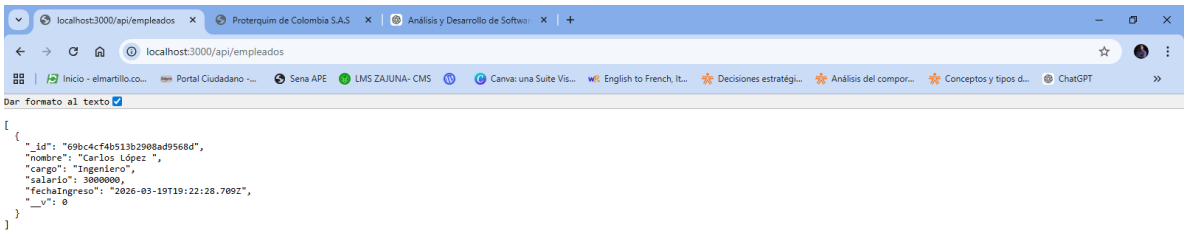
3. INTERFAZ DEL SISTEMA



Descripción:

Se presenta la interfaz del sistema desarrollada en Angular, la cual permite la interacción del usuario con el módulo de gestión de empleados.

4. SERVICIO API (JSON)



Descripción:

Se evidencia la respuesta del servicio API REST en formato JSON, demostrando la comunicación entre el backend y la base de datos.

5. PRUEBAS CRUD

Crear Empleado (CREATE)

Permite registrar un nuevo empleado en el sistema enviando los datos al servidor, donde son procesados y almacenados en la base de datos.

Método: POST

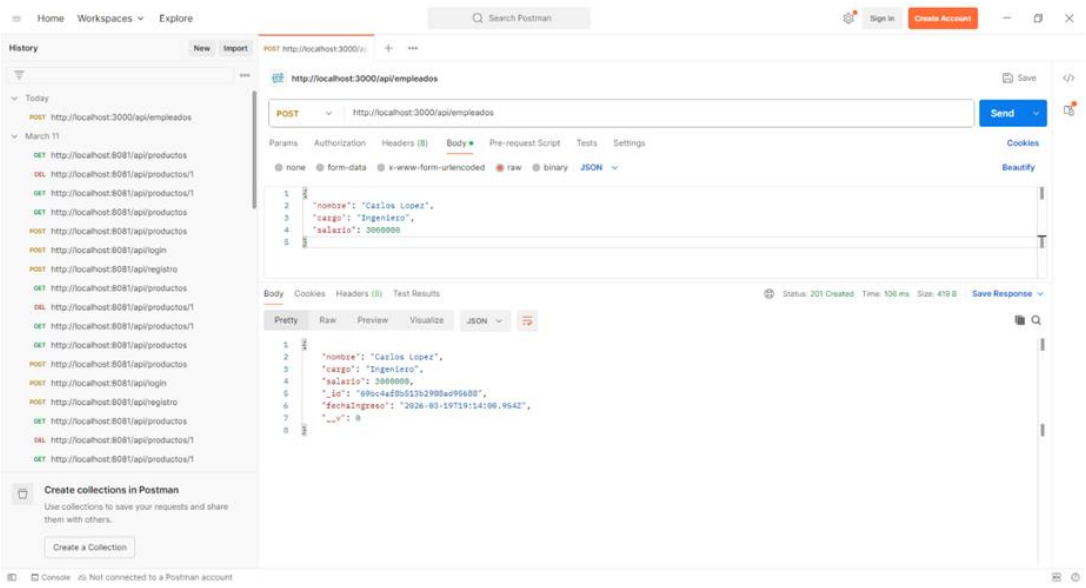
URL: http://localhost:3000/api/empleados

Body:

```
{
  "nombre": "Carlos López",
  "cargo" : "Ingeniero",
  "salario": 3000000
}
```

Resultado:

El empleado fue registrado correctamente en la base de datos y visualizado en la interfaz.



Listar Empleados (READ)

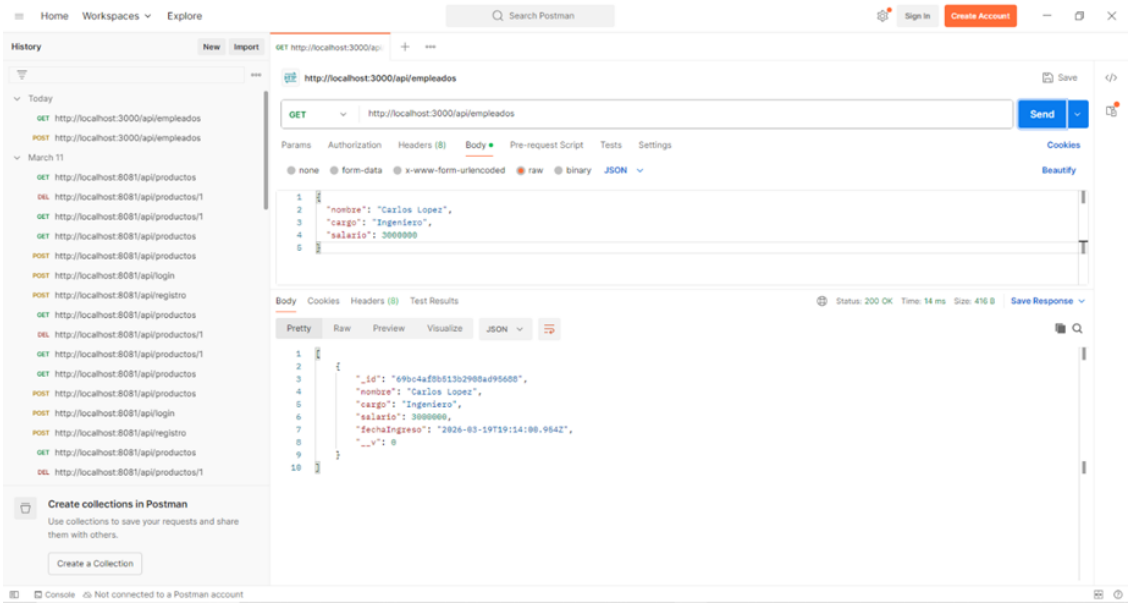
Permite consultar y obtener la lista de empleados registrados en el sistema, recuperando la información almacenada en la base de datos.

Método: GET

URL: http://localhost:3000/api/empleados

Resultado:

Se obtuvo correctamente el listado de empleados desde el backend y se visualizó en la tabla del sistema.



Actualizar Empleado (UPDATE)

Permite modificar la información de un empleado existente, enviando los datos actualizados al servidor para su respectiva actualización en la base de datos.

Método: PUT

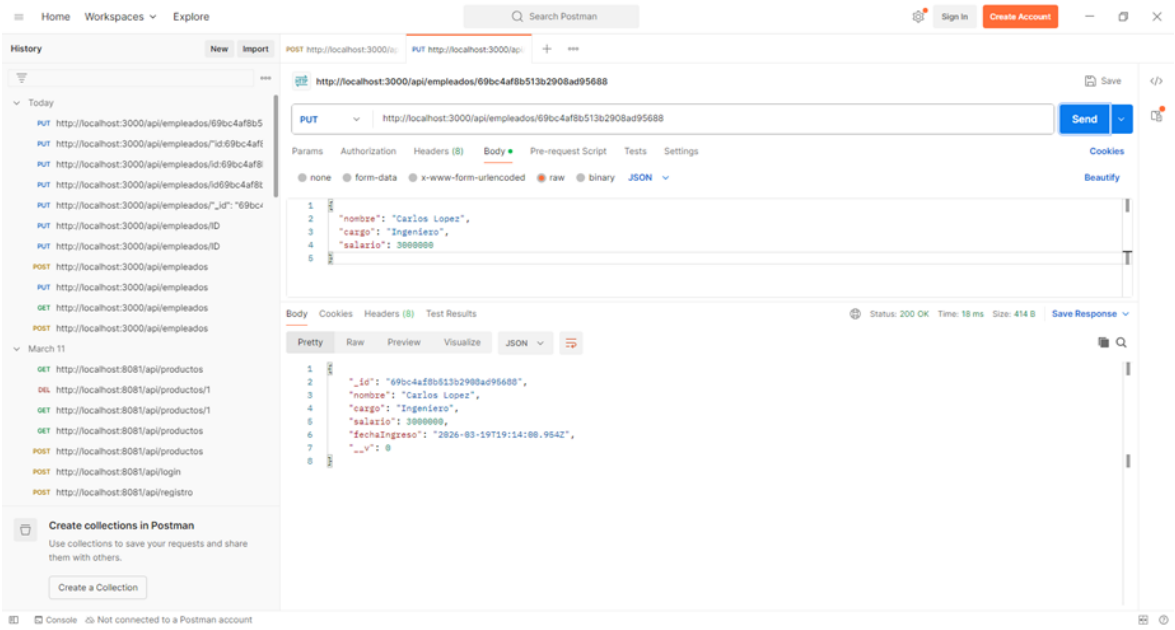
URL: http://localhost:3000/api/empleados/{id}

Body:

```
{
  "nombre": "Carlos Modificado",
  "cargo": "Ingeniero",
  "salario": 3500000
}
```

Resultado:

El empleado fue actualizado correctamente y los cambios se reflejaron en la interfaz.



Eliminar Empleado (DELETE)

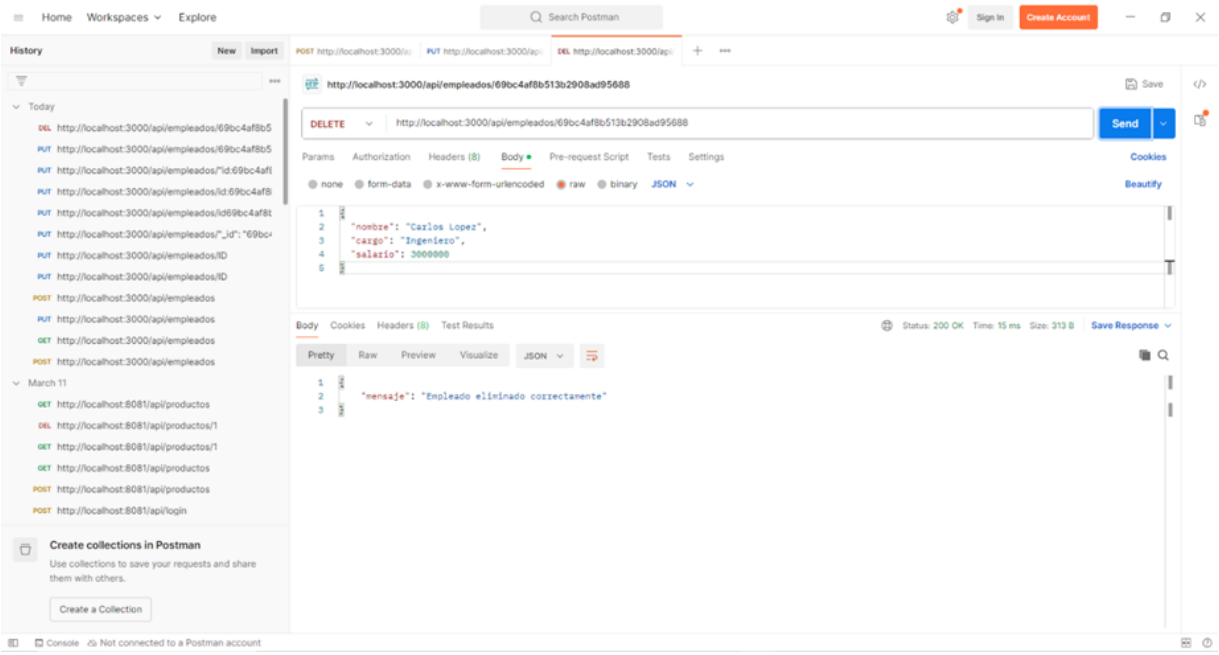
Permite eliminar un empleado del sistema mediante el envío de su identificador, removiendo el registro de la base de datos.

Método: DELETE

URL: http://localhost:3000/api/empleados/{id}

Resultado:

El empleado fue eliminado correctamente de la base de datos y dejó de mostrarse en la tabla.



16. DOCUMENTACIÓN DE DATOS DE ENTRADA Y SALIDA

Módulo: Gestión de Empleados

Método	Endpoint	Entrada	Salida	Descripción
POST	/api/empleados	nombre (string), cargo (string), salario (number)	Objeto del empleado creado y mensaje de confirmación	Permite registrar un nuevo empleado en el sistema
GET	/api/empleados	No requiere datos	Lista de empleados en formato JSON	Permite consultar todos los empleados registrados
PUT	/api/empleados/:id	ID del empleado, nombre (string), cargo (string), salario (number)	Objeto actualizado y mensaje de confirmación	Permite actualizar la información de un empleado existente
DELETE	/api/empleados/:id	ID del empleado	Mensaje de eliminación	Permite eliminar un empleado del sistema

Los datos de entrada y salida del sistema son gestionados mediante servicios REST, utilizando el protocolo HTTP y el formato JSON para el intercambio de información entre el frontend y el backend.

1. CONCLUSIONES

El desarrollo del sistema permitió aplicar conceptos de arquitectura modular, integración de componentes y desarrollo de aplicaciones web.

Se logró implementar un módulo funcional de gestión de empleados, así como definir la estructura de otros módulos del sistema, cumpliendo con los requerimientos de la evidencia.

El sistema desarrollado puede ser escalado para integrar nuevos módulos como usuarios, productos e inventarios, fortaleciendo su funcionalidad dentro de la empresa Proterquim.

2. BIBLIOGRAFÍA

- ✓ Angular Documentation. (s.f.). Recuperado de: https://angular.io
- ✓ Node.js Documentation. (s.f.). Recuperado de: https://nodejs.org
- ✓ Express.js Documentation. (s.f.). Recuperado de: https://expressjs.com
- ✓ MongoDB Documentation. (s.f.). Recuperado de: https://www.mongodb.com
- ✓ Git Documentation. (s.f.). Recuperado de: https://git-scm.com
- ✓ Pressman, R. (2014). Ingeniería del Software: Un enfoque práctico. McGraw-Hill.
- ✓ Sommerville, I. (2011). Ingeniería de Software. Pearson Educación.